PalmSource™ 99

Palm Computing® Platform Data Structures

David Fedor
Senior Developer Support Engineer

3Com
More connected.

Why Are We Here?

PalmSource 99

- Low-level explanation of the Palm OS™ database formats:
  - PRC
  - PDB
  - PQA
- We hadn’t documented it before

The Big Picture

- Files with .PRC extensions are resource DBs
- Files with .PDB and .PQA are record DBs
- “Database” is perhaps a grand term; they consist of:
  - Some header information
  - Sequential list of free-form chunks of memory (records or resources) each with a small amount of header information
- Currently the desktop (on-disk) format matches the handheld (in-memory) format pretty closely—but not exactly

PQAs

- PQAs are record databases (PDBs)
- PQAs have additional constraints, structures and definitions, used by Web Clipping and the Launcher
- More on their specifics later on in this talk

Official Documentation?

- PQA format documentation is ready
  - Documents just PQAs, but that's quite a bit
    - Includes 98% of PDB format, 95% of PRC format
- PRC, PDB documentation is in the works
  - DataMgr.h and DataPrv.h have all the structures
  - Plenty of 3rd party attempts at documentation and code
  - We wrote some code routines too; see the sample code area of http://www.palm.com/devzone/

Records Versus Resources

PalmSource 99

- Both are chunks of memory containing whatever your application wants
- Sometimes the system defines a resource or record format for things like MIDI sounds (records) or UI objects (resources) but other than that they’re free-form
- Record and resource DBs are really similar; only difference is the size of the header of each object

Records Versus Resources

- Resources have attributes:
  - Type (e.g. ‘tFRM’)
  - Creator (e.g. ‘psys’) (See Creator ID web page)
  - Memory Chunk (maintains its own size attribute)
- Records have attributes:
  - Delete
  - Dirty
  - Busy
  - Secret
  - Unique ID
  - Memory Chunk

Records Versus Resources

PalmSource 99

- Usage differs
  - Records are great for memos, address book entries…
  - Resources are great for Applications, UI objects…
- Both can be treated as ordered or unordered
  - Resources don't sort on the device
  - Records sort under your program's control

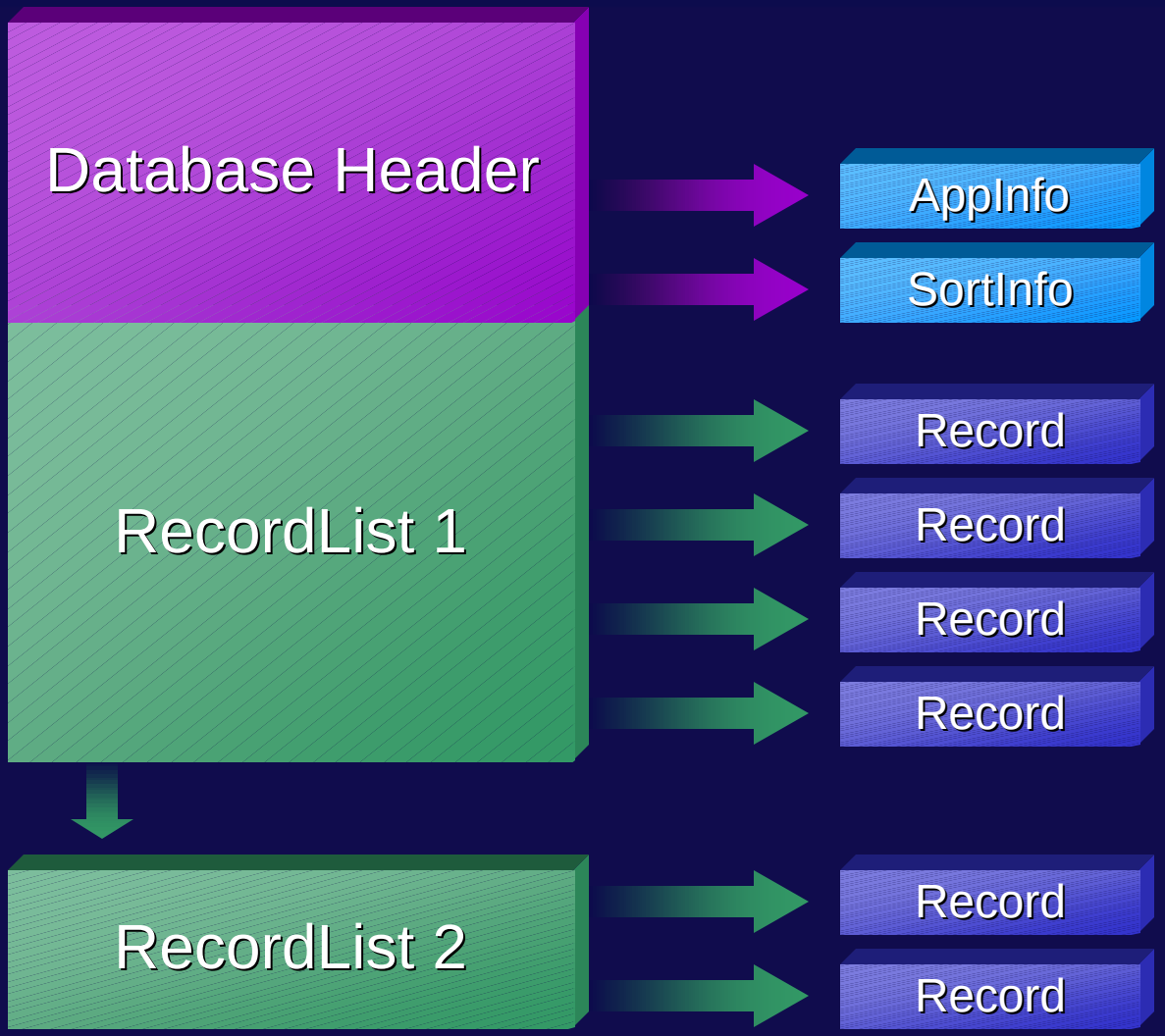
Accessing Entries

- Each entry (record or resource) can be accessed by index, or by searching on various of the attributes, but there's no extra indexing defined
- Searching is linear whereas accessing by index is O(1) (nearly)
- There's one or more RecordLists, each of which have pointers to several entries
  - So indexing isn't quite linear: has to walk the linked list of RecordLists to find the right one, then does an index in it
  - Ideally you only have one RecordList

LocalIDs

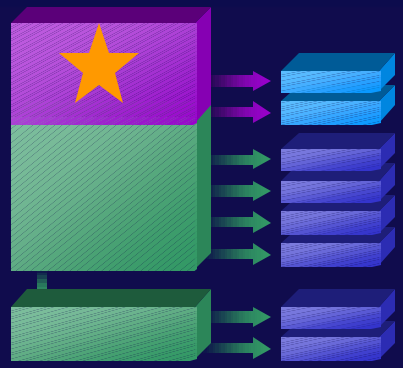
- A 4-byte, protected, card-relative handle used by the memory manager
- Used to refer to memory “chunks”—the basic allocation block of memory
- In memory, is the offset from the memory card’s base address
- In file formats, is the offset in bytes from the start of the file
- Used to link together all the pieces of memory that make up a database
- (See the memory manager paper and docs)

The Whole Picture



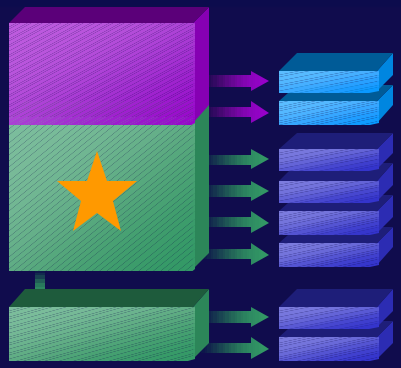
Database Header

- Database Header contents
  - Name (32 bytes)
  - Attributes (2)
  - Version (2)
  - CreationDate (4)
  - ModificationDate (4)
  - appInfoID (4)
  - sortInfoID (4)
  - type (4)
  - creator (4)
  - uniqueIDSeed (4)
  - RecordList (various lengths)



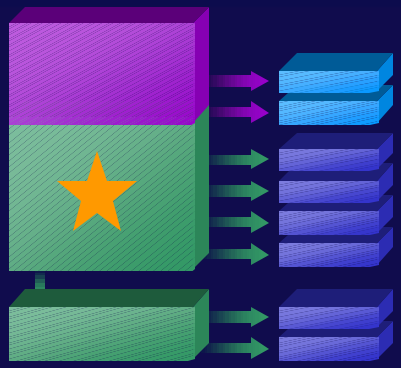
RecordList

- RecordList header
  - nextRecordListID (4 bytes)
  - numRecords (2)
  - padding (2)
- RecordEntry list
  - localChunkID (4)
  - recordAttributes (1)
  - uniqueID (3)
  - (repeat, numRecords times)

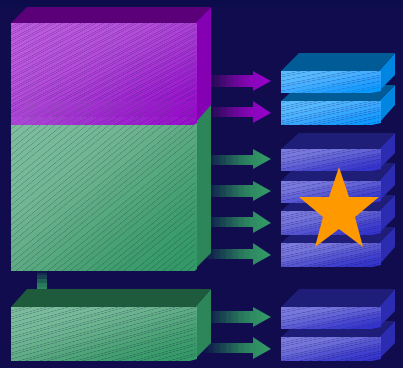


ResourceList

- RecordList header
  - nextRecordListID (4 bytes)
  - numRecords (2)
  - padding (2)
- ResourceEntry list
  - type (4)
  - id (2)
  - localChunkID (4)
  - (repeat, numRecords times)

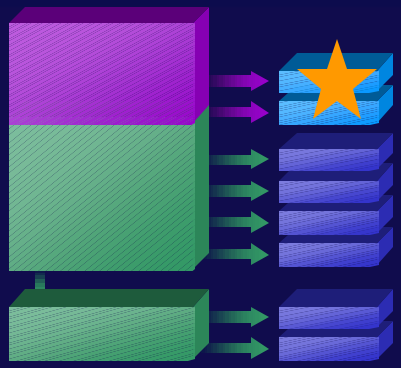


Record and Resource



- Just bytes that are under the program's control
- Notice no length attribute!
  - HotSync® technology, the Palm OS™ Emulator, DmCreateDatabaseFromImage, and other tools figure out the size of each object by subtracting localChunkIDs, when installing databases
  - The Memory Manager has overhead tracking the size of each chunk once it is installed

AppInfo and SortInfo



- Both are free-form…except for appInfo in record DBs that the launcher displays:
  - Signature ‘lnch’ (4 bytes)
  - hdrVersion (3) (2 bytes)
  - encVersion (for PQAs) (2 bytes)
  - verStrWords (2 bytes)—length of the version string, in words, which immediately follows
  - pqaTitleWords (2 bytes)—ditto for the title string
  - iconWords (2 bytes)—ditto for the large icon
  - smIconWords (2 bytes)—ditto for the small icon

PQA Web Content

PalmSource 99

- PQAs are Record databases with:
  - Standard database header
  - AppInfo block with icons, version information, user-visible title
  - One or more records, each containing a piece of “web content”—meaning a compressed HTML page or a bitmap
- Record contents are a defined format
  - (A very complex one)
  - (See the documentation)

PQA Record Contents

- Each record's data starts with:
  - urlOffset (4 bytes)—always set to 14
  - urlLength (2)
  - dataOffset, dataLength (4, 2)
  - contentType (1)
  - compressionType (1)
  - uncompressedDataSize (4)
  - flags (2) (always zero)
  - reserved (2) (always zero)
- Then the variable-length content
  - url string, then web content

PQA Web Content

- Many content types are defined, but only two are currently used
  - cmlContentTypeTextCml
  - cmlContentTypeImagePalmOS
- Compression types:
  - None (rarely used)
  - BitPacked (commonly used)
  - LZ77 (not yet used)
- This covers 68 documentation pages to explain fully; I'll use 7 slides to summarize

What Is CML?

- CML is *NOT* a new markup language, nor is it one-to-one with HTML
- It is a confusingly named compression scheme for HTML, better called the PQA Encoding format
- The term CML only exists in struct definitions and would have been too much of a pain to change, so we’re stuck with the name
- It refers to the compressed web content which you find in a PQA

What Is CML?

PalmSource 99

- A binary stream of data that used to be HTML
- An intermediate form of data in the Web Clipping world
- Never actually seen in real life; you instead see bit-packed CML
- Very complicated to read and debug and work with
  - “Readability and flexibility are compromised for compactness”
  - …and for use by the rendering engine

CML Example

- This HTML:

  <html><head><title>Example</title></head>
  <body>Body text</body></html>

- Generates this unpacked data:

```
45 78 61 6D 70 6C 65 00 42 6F 64 79
20 74 65 78 74 01 71
E  x  a  m  p  l  e  \0 B  o  d  y
sp t  e  x  t  cmlEnd
```

- …which then gets compacted further by taking out bits, generating “bit-packed” data

```
12 2F 4D 2A C5 40 12 15 13 E2 E5 5D
C8 5C 40
```

Bit-packing

PalmSource 99

Breakout of the bits from the previous page

```
00010 <single char escape>
01000101 E
11101 x
00110 a
10010 m
10101 p
10001 l
01010 e
00000 <title string textz
null terminator>
```

```
00010 <single char escape>
01000010 B
10100 o
01001 d
11110 y
00101 <space>
11001 t
01010 e
11101 x
11001 t
00001 <start of tag>
01110001 cmlTagCMLEnd
```

Tags and Bit-packing

- The data all starts as HTML, then gets simplified and packed down and compressed
- HTML tags map down to CML tags (many to fewer)
- By default CML objects are 5 bits, but can be many different lengths
  - E.g., tags are 8, integers can be 1 bit, 5 bits, 9…
- Integers pack down specially
  - E.g., IntV value 0 is one bit: 0.
  - IntV pattern 10 means 3 bits more coming, which can specify values -4 through 3

Tags

- Most tags are standalone, with parameters
  - cmlTagBGColor, followed by 3 bytes of color data
- Some turn on a mode and are turned off again
  - cmlTagTextBold, followed eventually by cmlCharEnd
- Some tags and modes can nest, some are terminated by a cmlCharEnd
- In each of them the bit-packing happens, and text can get compressed in multiple ways
- See the docs for all the full detail

Q&A


PalmSource™ 99
3Com
More connected.